

1 Что мы создаём

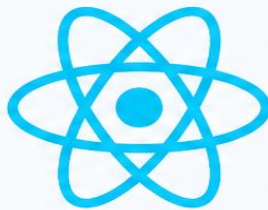
Цель: запустить простое React-приложение и понять назначение каждого инструмента.

В проекте будут:

- **React** — библиотека для интерфейса;
- **TypeScript** — проверка типов;
- **Webpack** — сборка проекта;
- **webpack-dev-server** — локальный сервер для разработки.



Готовые генераторы вроде Create React App или Vite **не используем**.



React



TypeScript



Webpack



webpack-dev-server

2

Создаём пустую папку

Создадим папку для проекта и перейдём в неё.

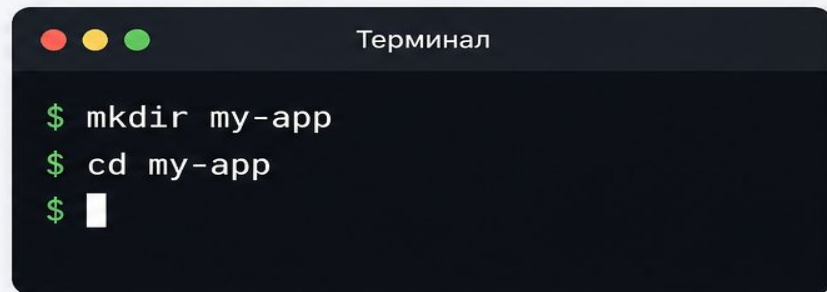
```
$ mkdir my-app  
$ cd my-app
```



my-app/

Что происходит:

- `mkdir my-app` — создаёт папку
- `cd my-app` — переходит внутрь неё
- Папка пока полностью пустая.
Все файлы и зависимости добавим вручную.



Теперь мы находимся в пустой папке `my-app`.
В терминале путь будет выглядеть так:

`.../my-app$`

3 Создаём package.json

Инициализируем проект и включаем ES-модули.

```
$ npm init -y  
$ npm pkg set type=module
```



package.json

Что делает команда:

- `npm init -y` — создаёт файл `package.json` с настройками по умолчанию.
- `npm pkg set type=module` — добавляет поле `"type": "module"` в файл `package.json`.

Это добавляет обязательное поле:

```
1 {  
2   "type": "module"  
3 }
```



Зачем нужно поле "type": "module"?

Оно позволяет использовать синтаксис ES-модулей в `webpack.config.js`:

```
1 import HtmlWebpackPlugin from "html-webpack-plugin";  
2  
3 export default {  
4   // настройки Webpack  
5 };
```

4 Устанавливаем React

Устанавливаем основные зависимости React.

```
$ npm install react react-dom
```

Что установлено:

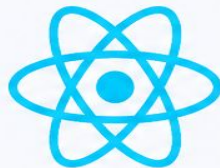
- **react** — библиотека для создания компонентов и управления состоянием.
- **react-dom** — отвечает за рендеринг React-компонентов в реальном DOM браузера.

После выполнения команды:

- 📁 Появится папка `node_modules/`
- 📄 Создастся файл `package-lock.json`
- 📄 В файле `package.json` появится раздел `dependencies`



Эти две зависимости — основа любого React-приложения. Без них компоненты не смогут отображаться в браузере.



react

создание компонентов



react-dom

вывод компонентов
в браузер

Фрагмент package.json:

```
1 {
2   "name": "my-app",
3   "version": "1.0.0",
4   "type": "module",
5   "dependencies": {
6     "react": "^18.3.1",
7     "react-dom": "^18.3.1"
8   }
9 }
```

React

React DOM

5 Устанавливаем Webpack

Устанавливаем инструменты для сборки и разработки.

```
$ npm install --save-dev webpack webpack-cli  
webpack-dev-server html-webpack-plugin
```

Назначение пакетов:

- **webpack** — собирает модули приложения в один или несколько файлов.
- **webpack-cli** — позволяет запускать Webpack из терминала.
- **webpack-dev-server** — запускает локальный сервер с автоматической пересборкой.
- **html-webpack-plugin** — создаёт HTML-файл и автоматически подключает собранный JavaScript.



Флаг **--save-dev** означает: эти пакеты нужны только для разработки и сборки, но не являются частью приложения.



После выполнения команды появятся папка `node_modules`, файл `package-lock.json` и раздел `devDependencies` в `package.json`.

Что делает каждый пакет:



webpack

Бандлер. Анализирует зависимости и собирает проект в результирующие файлы.



webpack-cli

Интерфейс командной строки для работы с Webpack.



webpack-dev-server

Локальный сервер для разработки. Поддерживает hot-reload и live-reload.



html-webpack-plugin

Генерирует HTML-файл и подключает к нему собранные скрипты автоматически.

6

Устанавливаем TypeScript

Добавляем TypeScript и связанные пакеты для работы с React.

```
$ npm install --save-dev typescript ts-loader  
@types/react @types/react-dom
```

Назначение пакетов:

- **typescript** — компилятор TypeScript.
Выполняет проверку типов и преобразует код.
- **ts-loader** — интеграция TypeScript с Webpack.
Передаёт файлы `.ts` и `.tsx` в компилятор.
- **@types/react** — содержит типы для React.
Позволяет TypeScript понимать код React.
- **@types/react-dom** — содержит типы для React DOM.
Добавляет типизацию для работы с DOM через React.



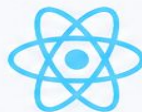
typescript

проверяет типы



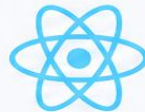
ts-loader

передаёт .ts и .tsx
файлы в TypeScript



@types/react

типы для React



@types/react-dom

типы для React DOM

Что появится в package.json:

Фрагмент package.json

```
1  {  
2    "devDependencies": {  
3      "typescript": "^5.4.0",  
4      "ts-loader": "^9.5.1",  
5      "@types/react": "^18.3.1",  
6      "@types/react-dom": "^18.3.0"  
7    }  
8  }
```



Почему --save-dev?

Эти пакеты нужны только для разработки и сборки.
Они не используются в браузере и не включаются в финальный бандл.

7 Создаём структуру проекта

Создаём папки и файлы, необходимые для работы приложения.

```
$ mkdir src public
$ touch src/index.tsx src/App.tsx public/index.html
  webpack.config.js tsconfig.json
```

Что создаёт команда:

- `mkdir src public` — создаёт две папки:
`src` и `public` — для исходников и статических файлов.
- `touch ...` — создаёт пустые файлы:
`src/index.tsx` `src/App.tsx` `public/index.html`
`webpack.config.js` `tsconfig.json`
— необходимые для запуска проекта.



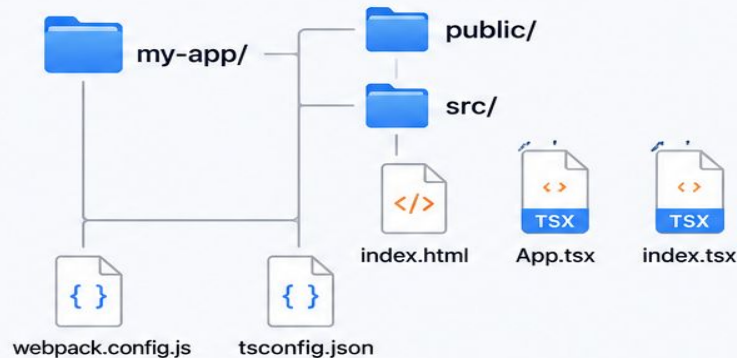
Зачем такая структура?

Разделение на папки помогает держать проект в порядке:

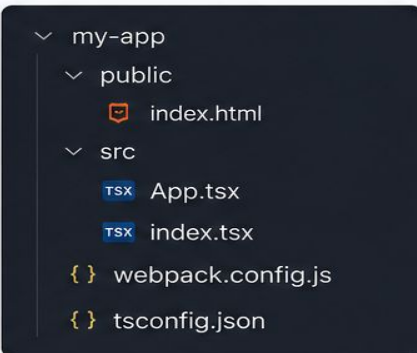
- `src` — весь исходный код приложения.
- `public` — статические файлы, доступные как есть.

Это стандартный подход, который используют многие проекты.

Структура проекта:



Вид в проводнике (пример):



Папки обозначены значком папки.



HTML-файл находится в папке `public`.



TypeScript-файлы с JSX имеют расширение `.tsx`.



Конфигурационные файлы проекта находятся в корне.

8 Добавляем HTML-шаблон

Создаём базовый HTML-файл для приложения.

Файл: `public/index.html`

Содержимое файла:

```
1 <!DOCTYPE html>
2 <html lang="ru">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width,
6       initial-scale=1.0" />
7     <title>React + Webpack + TypeScript</title>
8   </head>
9   <body>
10     <div id="root"></div>
11   </body>
12 </html>
```



Почему это важно?

Этот файл — шаблон для `html-webpack-plugin`.
Плагин возьмёт его, вставит в него собранный `bundle.js`
и создаст итоговый HTML в папке `dist/`.

Ключевые элементы:



`<!DOCTYPE html>`

Стандартный тип документа HTML5.



`<div id="root"></div>`

Контейнер, в который React будет
выводить приложение.



`html-webpack-plugin`

Автоматически подключит скрипт
`bundle.js` перед закрывающим `</body>`.

Что происходит дальше?



В итоговый HTML будет автоматически вставлен собранный JavaScript.

9 Создаём React-компонент

Создаём простой компонент App, который отобразит заголовок.

Файл: `src/App.tsx`

Содержимое файла:

```
1  const App = () => {  
2    return <h1>Hello, React!</h1>;  
3  };  
4  
5  export default App;
```

Что здесь происходит:

- 1 Создаём функциональный компонент `App`.
- 2 Компонент возвращает JSX-разметку — заголовок `<h1>`.
- 3 Компонент экспортируется по умолчанию, чтобы его можно было импортировать в других файлах.



Почему .tsx?

Расширение `.tsx` используется для TypeScript-файлов, которые содержат JSX-разметку.

Что такое компонент?



Компонент — это независимая часть интерфейса, которая может использоваться повторно.

Компоненты делают код понятнее, удобнее для поддержки и масштабирования.

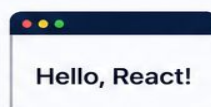
Как используется дальше?



App.tsx
(компонент)



index.tsx
(точка входа)



Браузер
(пользователь видит результат)



Совет:

На этом этапе у нас есть первый компонент. Дальше мы создадим точку входа и отобразим его в браузере.

10 Добавляем точку входа

Создаём файл, с которого начинается работа React-приложения.

Файл: `src/index.tsx`

Содержимое файла:

```
1 import { createRoot } from "react-dom/client";
2 import App from "./App";
3
4 createRoot(document.getElementById("root")!).render(<App />);
5
```

Что здесь происходит:

- 1 Ищем корневой элемент в HTML:
`document.getElementById("root")` — находит элемент с `id="root"`.
Знак `!` сообщает TypeScript, что элемент точно существует.
- 2 Создаём корень React:
`createRoot(...)` — инициализирует корневой React-элемент.
- 3 Рендерим компонент:
`render(<App />)` — выводим компонент `<App />` в корень приложения.



Почему файл называется `index.tsx`?

Это стандартное название точки входа в React-приложения.
В `webpack.config.js` мы укажем этот файл как `entry`.

Импорты:



`react-dom/client`

Современный API React для работы с DOM.
Функция `createRoot` рекомендуется начиная с React 18.



`./App`

Импортируем наш компонент `App`
из файла `App.tsx`.

Как это работает:



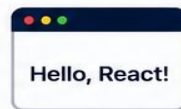
`index.tsx`
(точка входа)



React
(создаём корень
и рендерим)



`App.tsx`
(компонент)



Браузер
(пользователь видит
результат)



Важно:

Элемент с `id="root"` должен существовать в HTML,
иначе приложение не сможет отрендериться.

Создаём файл `tsconfig.json`, который управляет работой TypeScript.

Файл `tsconfig.json`:

```
1  {
2    "compilerOptions": {
3      "target": "ES2020",
4      "module": "ESNext",
5      "moduleResolution": "Bundler",
6      "jsx": "react-jsx",
7      "strict": true,
8      "esModuleInterop": true,
9      "sourceMap": true,
10     "skipLibCheck": true
11   },
12   "include": ["src"]
13 }
```



include

```
"include": ["src"]
```

TypeScript будет анализировать только файлы внутри папки `src`.

Основные настройки:

```
"target": "ES2020"
```

Версия JavaScript, которую будет генерировать TypeScript.

```
"module": "ESNext"
```

Используем современную систему ES-модулей.

```
"moduleResolution": "Bundler"
```

Поиск импортов выполняется так же, как это делает Webpack.

```
"jsx": "react-jsx"
```

Позволяет использовать JSX без импорта React в каждом файле.

```
"strict": true
```

Включает максимальную проверку типов.

```
"esModuleInterop": true
```

Упрощает импорт CommonJS-пакетов.

```
"sourceMap": true
```

Позволяет видеть исходный TypeScript-код в DevTools браузера.

```
"skipLibCheck": true
```

Не проверяет типы внутри сторонних библиотек.



`src/*.tsx`



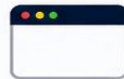
TypeScript



JavaScript



Webpack



Browser

Создаём файл `webpack.config.js` — конфигурацию для сборки проекта.

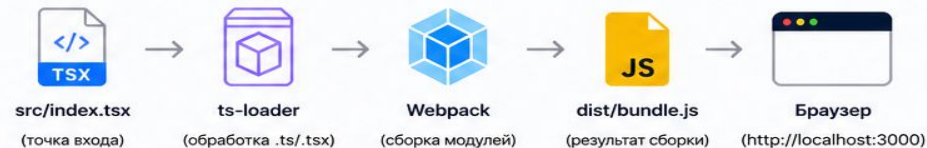
Файл `webpack.config.js`:

```
1 import HtmlWebpackPlugin from "html-webpack-plugin";
2
3 export default {
4   entry: "./src/index.tsx",
5
6   output: {
7     filename: "bundle.js",
8     clean: true
9   },
10
11   resolve: {
12     extensions: [".tsx", ".ts", ".js"]
13   },
14
15   module: {
16     rules: [
17       {
18         test: /\.tsx?$/,
19         use: "ts-loader",
20         exclude: /node_modules/
21       }
22     ]
23   },
24
25   plugins: [
26     new HtmlWebpackPlugin({
27       template: "./public/index.html"
28     })
29   ],
30
31   devServer: {
32     port: 3000,
33     open: true
34   }
35 };
```

Что делает эта конфигурация:

entry: <code>"./src/index.tsx"</code>	Указывает точку входа приложения.
output	Результат сборки будет в папке <code>dist</code> в файле <code>bundle.js</code> . Опция <code>clean: true</code> очищает папку перед новой сборкой.
resolve.extensions	Позволяет импортировать файлы без указания расширения (например, импортировать компонент как <code>./App</code>).
module.rules	Файлы с расширением <code>.ts</code> и <code>.tsx</code> обрабатываются через <code>ts-loader</code> , который передаёт их в TypeScript.
plugins	<code>HtmlWebpackPlugin</code> создаёт итоговый HTML-файл на основе шаблона из <code>public/index.html</code> и автоматически подключает <code>bundle.js</code> .
devServer	Запускает локальный сервер разработки на порту 3000 и автоматически открывает браузер.

Как работает сборка:



Важно:

Webpack по умолчанию складывает результат в папку `dist`, поэтому абсолютный путь указывать не требуется.

13 Добавляем команды запуска

Настраиваем раздел `scripts` в файле `package.json`.

Обновляем `package.json`:

```
1  {
2    "name": "my-app",
3    "version": "1.0.0",
4    "type": "module",
5    "scripts": {
6      "start": "webpack serve --mode development",
7      "build": "webpack --mode production"
8    },
9    "dependencies": {
10     "react": "^...",
11     "react-dom": "^..."
12   }
13 }
```



Что получилось:

- `"type": "module"` — включает ES-модули для `webpack.config.js`.
- `"start"` — запускает режим разработки с локальным сервером.
- `"build"` — создаёт production-сборку в папку `dist`.



Проверка команд

Вы можете проверить, что команды добавлены правильно:

Доступные команды

Команда	Описание
<code>npm start</code>	Запускает <code>webpack-dev-server</code> в режиме разработки (<code>development</code>).
<code>npm run build</code>	Создаёт оптимизированную сборку для продакшена (<code>production</code>) в папку <code>dist</code> .



Важно

Файл `webpack.config.js` использует синтаксис ES-модулей (`import/export`). Для этого необходимо поле `"type": "module"`.



Итого

Теперь у нас есть две основные команды:

- `npm start` — разработка и тестирование приложения;
- `npm run build` — финальная сборка для публикации.

`npm run` # или `npm run build --dry-run`

14 Добавляем .gitignore

Создаём файл `.gitignore` в корне проекта.

Файл `.gitignore`:

```
1 node_modules
2 dist
```



Что такое `.gitignore`?

Файл `.gitignore` сообщает Git, какие файлы и папки не нужно отслеживать и добавлять в репозиторий.



Проверяем

После создания файла выполните:

```
git status
```

В выводе не должно быть папок `node_modules` и `dist`.

Почему эти папки игнорируются?



`node_modules`

Содержит все установленные зависимости.
Эту папку можно восстановить командой `npm install`.
Хранить её в Git не нужно.



`dist`

Папка со сгенерированной сборкой проекта.
Она создаётся командой `npm run build`.
Её можно пересоздать в любой момент.

Структура проекта с `.gitignore`

```
my-app/  
|-- node_modules/  # игнорируется  
|-- dist/          # игнорируется  
|-- public/  
|-- src/  
|-- .gitignore  
|-- package.json  
|-- tsconfig.json  
|-- webpack.config.js
```



В репозиторий
попадут только нужные
исходные файлы.

14 Добавляем .gitignore

Создаём файл `.gitignore` в корне проекта.

Файл `.gitignore`:

```
1 node_modules
2 dist
```



Что такое `.gitignore`?

Файл `.gitignore` сообщает Git, какие файлы и папки не нужно отслеживать и добавлять в репозиторий.



Проверяем

После создания файла выполните:

```
git status
```

В выводе не должно быть папок `node_modules` и `dist`.

Почему эти папки игнорируются?



`node_modules`

Содержит все установленные зависимости.
Эту папку можно восстановить командой `npm install`.
Хранить её в Git не нужно.



`dist`

Папка со сгенерированной сборкой проекта.
Она создаётся командой `npm run build`.
Её можно пересоздать в любой момент.

Структура проекта с `.gitignore`

```
my-app/
|-- node_modules/  # игнорируется
|-- dist/          # игнорируется
|-- public/
|-- src/
|-- .gitignore
|-- package.json
|-- tsconfig.json
|-- webpack.config.js
```



В репозиторий
попадут только нужные
исходные файлы.

15 Проверяем типы

TypeScript позволяет проверить код на ошибки типов без запуска сборки.

Запускаем команду:

```
$ npx tsc --noEmit
```

i Опция `--noEmit` означает «не создавать файлы». TypeScript выполнит только проверку типов.

✓ Ошибок нет

Команда завершилась без вывода.
Ваш код корректен с точки зрения типов.

```
$ npx tsc --noEmit
$
```



Совет

Регулярно запускайте эту команду во время разработки, чтобы находить ошибки как можно раньше.



Важно

Проверка типов не запускает сборку и не создаёт папку `dist`. Это быстрая проверка вашего TypeScript-кода.

Что происходит при выполнении:

- 1 TypeScript читает файл `tsconfig.json`.
- 2 Проверяет все файлы из папки `src` на соответствие типам.
- 3 Если ошибки есть — выводит их в терминал.
- 4 Если ошибок нет — команда завершается без вывода.



Есть ошибки типов

TypeScript выведет список ошибок с указанием файлов и строк. Исправьте их и запустите команду снова.

```
$ npx tsc --noEmit
src/App.tsx:5:10 - error TS2322: Type 'string' is not assignable
to type 'number'.

5   const count: number = "hello";
                        ~~~~~

Found 1 error.
```



TypeScript
(проверка типов)



Ошибок нет
(успех)



Ошибки есть
(исправить и повторить)

16 Запускаем режим разработки

Запускаем локальный сервер для разработки с автоматической пересборкой.

Запускаем команду:

```
$ npm start
```

 Команда `npm start` выполняет скрипт из `package.json`:

```
1 {  
2   "scripts": {  
3     "start": "webpack serve --mode development",  
4     "build": "webpack --mode production"  
5   }  
}
```



Что происходит:

- Webpack собирает проект в памяти.
- Запускается локальный сервер.
- Браузер автоматически открывается.
- При изменении файлов проект пересобирается автоматически.

В результате:



Терминал:

```
INFO webpack-dev-server v4.15.1  
  
Project is running at:  
> Local: >: http://localhost:3000/  
> On Your Network: http://192.168.1.10:3000/  
  
Content not from webpack is served from 'dist' directory  
  
i [webpack-dev-server] Live Reloading enabled.
```



Полезные советы

- Если браузер не открылся — откройте `http://localhost:3000` вручную.
- Если порт занят, измените порт в `webpack.config.js` в разделе `devServer`.

- По умолчанию сервер работает на порту 3000.
- Вы можете изменить порт в конфигурации.



17 Создаём production-сборку

Создаём оптимизированную сборку проекта для размещения на сервере.

Запускаем команду:

```
$ npm run build
```

Что происходит:

- 1 Webpack запускается в режиме `production`.
- 2 Код минифицируется и оптимизируется.
- 3 Все модули объединяются в файлы.
- 4 Результат помещается в папку `dist`.

Пример вывода в терминале:

```
$ npm run build
> my-app@1.0.0 build
> webpack --mode production

asset bundle.js 180 KiB [emitted] [minimized] (name: main)
asset index.html 187 bytes [emitted]
orphan modules 123 KiB [orphan] 45 modules
runtime modules 1.02 KiB 5 modules
cacheable modules 123 KiB

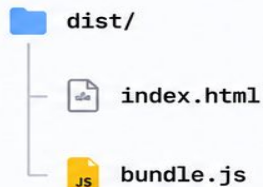
webpack 5.91.0 compiled successfully in 2345 ms
```



Итог

Команда `npm run build` создаёт готовую **production-сборку** в папке `dist`. Эту папку можно выложить на любой статический хостинг или сервер.

В результате создаётся папка `dist`:



Папка со всеми файлами готовой сборки.

HTML-файл с подключённым скриптом `bundle.js`.

Минифицированный JavaScript-файл с кодом приложения.

Размер и оптимизация



- Код минифицирован (удалены лишние пробелы, комментарии).
- Имена переменных и функций сокращены.
- Убраны неиспользуемые модули (tree shaking).
- Файлы готовы для публикации на сервере или хостинге.



Важно

- Папку `dist` нужно загружать на сервер целиком.
- Никаких исходных файлов `src`, `tsconfig.json` и т.д.) на сервере быть не должно.
- Эта сборка не предназначена для разработки.



18 Итог

✓ Мы вручную прошли полный путь от пустой папки до работающего React-приложения!
Без генераторов и скрытых настроек — каждый файл и пакет добавлен осознанно.

Что мы сделали:

- 1 Создали пустую папку.
- 2 Создали `package.json`.
- 3 Установили React.
- 4 Добавили Webpack.
- 5 Подключили TypeScript.
- 6 Создали HTML-шаблон и React-компоненты.
- 7 Настроили сборку (TypeScript и Webpack).
- 8 Проверили типы.
- 9 Запустили dev-сервер.
- 10 Собрали production-версию.

Наш стек



React
библиотека
для интерфейса



TypeScript
проверка
типов



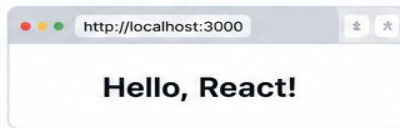
Webpack
сборка
проекта



webpack-dev-server
локальный сервер
для разработки

Что у нас получилось

```
$ npm start
```



Режим разработки

```
$ npm run build
```



Production-сборка



Главное

Теперь вы понимаете, зачем нужен каждый файл и пакет в проекте, как работает сборка и как из исходного кода получается готовое приложение. Это основа, на которую можно строить более сложные проекты.

```
# Всё готово!  
npm start # разработка  
npm run build # production-сборка
```

